Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Webová aplikace s texty Bible

2026

Patrik Prinz

Vedoucí práce:
Mgr. Tomáš Urbanec, Ph.D.

Studijní program: Informatika,
Specializace: Programování a vývoj software

Bibliografické údaje

Autor:	Patrik Prinz
Název práce:	Webová aplikace s texty Bible
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet stran:	15
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	český

Bibliographic info

Author:	Patrik Prinz
Title:	Web application with Bible texts
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	15
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Czech

Anotace

Ukázkový text závěrečné práce na Katedře informatiky Přírodovědecké fakulty Univerzity Palackého v Olomouci, který je zároveň dokumentací stylu pro text práce v LaTeXu. Zdrojový text v LaTeXu je doporučeno použít jako šablonu pro text skutečné závěrečné práce studenta.

Synopsis

Sample text of thesis at the Department of Computer Science, Faculty of Science, Palacký University Olomouc and, at the same time, documentation of the LaTeX style for the text. The source text in LaTeX is recommended to be used as a template for real student's thesis text.

Klíčová slova: webová aplikace; Bible; PWA; TypeScript; DevOps; CI/CD

Keywords: web application; Bible; PWA; TypeScript; DevOps; CI/CD

Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1 Úvod **7**

2 Podobné projekty **8**
2.1 Časoslov.sk . . . 8
2.2 BibleGateway . . . 8
2.3 Beblia.com . . . 8

3 Komponenty projektu **8**
3.1 Monorepozitář . . . 9
3.1.1 Společná konfigurace . . . 9
3.1.2 Použité nástroje pro správu projektu . . . 9
3.1.2.1 GIT . . . 9
3.1.2.2 pnpm . . . 10
3.2 Server . . . 10
3.2.1 Architektura . . . 10
3.2.1.1 Struktura modulu . . . 11
3.3 Frontend . . . 11
3.4 Initializer . . . 11

4 CI/CD **11**
4.1 Verzování . . . 11
4.1.1 Větve . . . 12
4.1.2 Commity . . . 12
4.2 CI/CD pipeline . . . 13

5 Použité technologie **13**
5.1 TypeScript . . . 13
5.2 Express . . . 14
5.3 Vue.js . . . 14
5.4 Elastic search . . . 14
5.5 Docker . . . 14
5.6 Github actions . . . 14

Seznam zkratek **15**

Seznam tabulek

1 Úvod

Cílem této práce je vytvořit aplikaci pro použití v Řeckokatolické farnosti v Olomouci. Farnost je v současnosti vícenárodnostní a v posledních letech se značně rozrůstá. nejpočetněji jsou zde zastoupeni farníci z Ukrajiny. Texty Bible jsou v katolické církvi velmi často využívané při liturgických obřadech, ale také při osobní modlitbě a rozjímání.

Pro Apoštolský Exarchát České Republiky, který je nadřazenou organizací Olomoucké farnosti existuje kalendář, který obsahuje i biblické texty pro každý den v roce. Biblické texty v církevních obřadech jsou často vázány ke svátkům, které jsou buď pevně svázány s datem, nebo jsou pohyblivé a připadají každý rok na jiné datum. Mnoho textů je také svázáno s konkrétním obřadem, nebo událostí (např. křest, svěcení domu...).

Texty z knihy žalmů jsou často používány jako samostatné modlitby. Východní církve žalmy rozdělují do 20 skupin zvaných kaftizmy, které obsahují většinou okolo 4-8 žalmů. Ke kaftizmám jsou přidruženy další texty, které se modlí zároveň s nimi.

Farnost má vlastní společenství, které se modlí modlitby žalmů a funguje od roku 2024. Každý člen společenství se modlí jednu z kaftizem během týdne. Kaftizmy se mezi členy společenství pravidelně střídají.

Nenašel jsem dosud žádný projekt, který by poskytoval možnosti práce s biblickými texty v tomto rozsahu. Proto chci vytvořit aplikaci, která by sdružovala texty více jazyků s možností vlastní správy a přidávání dalších překladů a textů pro vlastní četbu a studium. S tímto bude spojena i možnost udržování vlastního kalendáře který bude obsahovat zejména úryvky na každý den. Také by zde měla být obsažena možnost vytvářet si a ukládat vlastní úryvky pro vlastní použití, nebo sdílení ve skupině. Proces rozdělení kaftizem v rámci společenství žaltáře plánuji zčásti zautomatizovat a zároveň poskytnout na stejném místě i texty samotné i s přidruženými modlitbami spolu s možností označit, že se člen již kaftizmu pomodlil.

Dále bych chtěl, aby platforma byla do budoucna dále rozšiřitelná a bylo možné přidávat další funkcionality a aby mohl projekt i v budoucnu reflektovat požadavky společenství, které jej bude využívat a pravděpodobně se i dále rozvíjet.

Dále se chci v této práci zaměřit na průzkum a využití konceptů z oblasti DevOps, které usnadní vývoj samotný v dlouhodobém měřítku, urychlí integraci a nasazení aplikace. Na projektu pracuji sám, proto je pro mne výhodné především zautomatizovat proces ověření a nasazení software a sledování jeho stavu. I za cenu větší náročnosti přípravy a počáteční konfigurace je pro mne výhodné získat značně větší spolehlivost a rychlost nasazení a také do budoucna usnadní řadu problémů s udržováním potřebné infrastruktury. Díky této filosofii se mohu i v jednočlenném týmu, po nasazení potřebné infrastruktury, soustředit na svou roli programátora bez nutnosti stále ručně udržovat vše potřebné pro chod projektu.

2 Podobné projekty

V rámci této práce jsem hledal a procházel i další software zaměřený na práci s texty Bible i dalšími liturgickými texty. Aplikací s tímto zaměřeních existuje celá řada. Většina je však určena k jiným účelům a především pro osobní použití, ne pro použití ve farnosti samotné, případně v jiném společenství. Podobné projekty bývají zaměřeny na samotné čtení a studium biblických textů. Většina z projektů má také uzavřený zdrojový kód a proto je nelze upravit a přizpůsobit pro specifické potřeby, nebo je již zastaralá a dlouhodobě neudržovaná. Dále existuje software zaměřený na texty modliteb samotné. Ty však většinou obsahují pouze samotné modlitby, typicky zobrazované podle dne. Projekt, který vyvíjím v rámci této práce sdružuje a implementuje více specifických požadavků, které ve většině případů nespadají do stejné domény. Pro případ chodu farnosti, nebo jiného společenství je však výhodné je sdružit do jednoho projektu. V seznamu níže uvádím některé z projektů, se kterými jsem se dosud setkal.

2.1 Časoslov.sk

Časoslov.sk je projekt vyvíjený Radou pre mládež Košickej eparchie na slovensku. Poskytuje texty pro jednotlivé dny v roce. Nabízí webovou i mobilní aplikaci s obdobnými funkcionalitami. Časoslov.sk se zaměřuje především na samotné texty pro jednotlivé dny. Neposkytuje možnost samostatného čtení a studia biblických textů, správu více překladů, ani funkcionalitu skupin.

2.2 BibleGateway

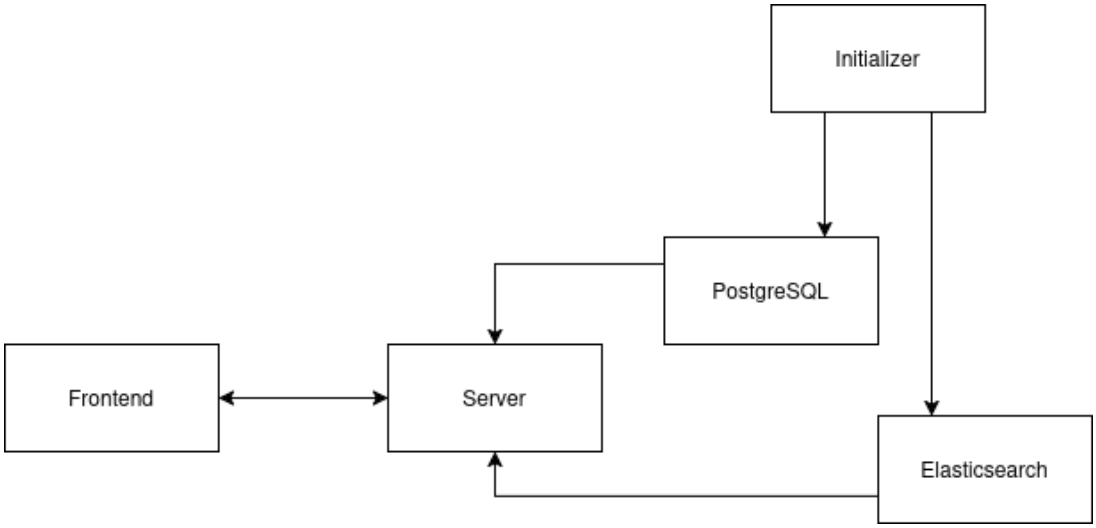
BibleGateway je Webová aplikace sloužící ke studiu biblických textů. Její doména se omezuje pouze na samotné procházení textů Bible a neimplementuje funkcionality jako modlitení skupiny, nebo žaltář a další modlitby. Aplikace na mnoha místech nabízí uživateli placenou verzi, což působí rušivě a není v souladu s mými požadavky.

2.3 Beblia.com

Beblia.com je další webová aplikace, která nabízí čtečku Biblických textů. Obsahuje také velké množství překlaů a v rámci projektu je k dispozici i databáze různých překladů v XML formátu. Texty však nemají přiloženou žádnou licenci, takže není možné je volně používat bez souhlasu autora.

3 Komponenty projektu

Projekt je složen z 5 hlavních komponent. Orchestrace a propojení komponent v podobě docker kontejnerů je realizována pomocí docker compose. V rámci práce jsem vytvořil 3 z nich.



Obrázek 1: Blokové schéma

3.1 Monorepozitář

Kontejnery tvořící projekt se vytvářejí z kódu a konfigurací uložených v monorepozitářové struktuře. Data jednotlivých kontejnerů jsou umístěna v příslušných balíčcích, které jsou reprezentovány podadresáři s příslušnými názky v adresáři packages.

Hlavní výhodou struktury monorepozitáře je přirozené sdružení více balíčků do jednoho projektu. To umožňuje jednodušší build proces a vydávání nových verzí. Dále je výhodná i možnost sdílení konfigurace a některých závislostí. Díky společnému umístění v projektu lze velmi jednoduše spustit celý projekt v testovacím prostředí a ověřit správnou komunikaci balíčků mezi sebou a případné odstranění nalezení chyb. Projekt je také velmi jednoduše přenositelný a pro konfiguraci na novém zařízení vyžaduje naprosto minimální konfiguraci.

3.1.1 Společná konfigurace

Každý balíček má vlastní specifické konfigurace. Společné konfigurace, které jsou globální pro projekt, nebo sdílené více balíčky, jsou umístěny v hlavním adresáři projektu.

S polečná konfigurace zahrnuje především docker compose soubory, konfiguraci nástrojů pro kontrolu kvality a spoelčnou konfiguraci pro TypeScript.

3.1.2 Použité nástroje pro správu projektu

3.1.2.1 GIT Nástroj GIT je v projektu použit pro správu verzí projektu. V CI/CD je verzování klíčové a řídí procesy kontroly kvality a doručení připravených artefaktů uživateli. Více se budu konvencím verzování v projektu věnovat dále v textu.

3.1.2.2 pnpm Pnpm jsem použil pro správu zavislostí projektu a pro implementaci onom skriptů, což jsou příkazy uložené v konfiguračních souborech pnpm. Umožnují jednoduché spouštění často opakovaných příkazů. Tuto možnost jsem využil pro zjednodušení přísrupu k nástrojům pro kontrolu kódu, kompilaci a spuštění docker compose.

Pnpm je správce balíčků pro node js. Pro tento správce balícků jsem se rozhodl, neboť jsem byl z dřívějších projektů zvyklý na použití npm. Pnpm má obdobné rozhraní, ale poskytuje mnoho optimalizací rychlosti instalace balíčků a využití místa na disku. Dále také poskytuje možnost správy zavislostí vnořených balíčků, což značně usnadňuje práci s monorepozitářem.

3.2 Server

Server poskytuje HTTP API pro manipulaci s daty uloženými v elastic search a PostgreSQL databázích. Poskytuje možnost správy uživatelů, sezení, kontrolu oprávnění a implementaci business logiky.

3.2.1 Architektura

Klíčovým aspektem při návrhu architektury pro mě byla budoucí rozšířitelnost aplikace. Proto jsem se rozhodl pro využití modulární monolitické architektury, která aplikaci rozděluje do více modulů, které reprezentují jednotlivé části aplikace podle částí domény (např. správa skupin, texty Bible, ...). Moduly jsou od sebe odděleny, což usnadňuje vývoj samotný a usnadňuje udržení přehledu o dopadech provedených změn na zbytek projektu.

Modulární monolit je kompromisem mezi mikroservisy a tradiční monolitickou architekturou. Stejně jako architektura mikroservisů, rozděluje aplikaci do více celků (v tomto případě jsou to moduly). Moduly sdružují funkcionalitu spjatou s danou částí domény a pro ostatní moduly poskytují jasně definované rozhraní. Narozdíl od mikrservisů však není aplikace rozdělena do více servisů, což usnadňuje počáteční nastavení a nasazení projektu spolu s jednodušším laděním a integrací celku.

Narozdíl od monolitické architektury je modulární monolit časově náročnější na úvodní konfiguraci, tuto nevýhodu však vyvažuje skutečnost, že v modulární struktuře je možné se podstatně přirozeněji orientovat, protože struktura aplikace respektuje zvolenou doménu. Toto je výhodou zvláště u rozsáhlejších projektů, i přesto jsem však tuto architekturu zvolil i v tomto projektu, který je malého rozsahu. S ohledem na budoucí možnosti rozšíření se projekt takto může dále rozrůstat s podstatně menším ztížením orientace.

Jádrem aplikace je sada globálně dostupných nástrojů, které implementují funkcionality, které jsou nezávislé na doméně a používají se na více místech napříč aplikací.

Patří sem například middleware, který zajišťuje valitaci dat, ošetření chyb a logování. Dále také sdílené knihovny, jako například adaptéry pro PostgreSQL a Elastic Search.

Moduly

3.2.1.1 Struktura modulu Moduly sdružují funkcionalitu, která je spojena na úrovni domény projektu. Každý modul obsahuje:

- **Router** mapuje HTTP požadavky na příslušné metody controlleru. Prostřednictvím middlewaru se zde také provádí ověření, jestli je uživatel přihlášený a provádí se validace dat požadavku.
- **Controller** validuje a zpracovává data a požadavky. Zajišťuje komunikaci s routerem a spouští service. Controller se stará o úlohy specifické pro exporess. Extrahuje data z Request objektu a nastavuje data, která jsou odeslána zpět uživateli.
- **service:** zajišťuje získání a zpracování dat na základě požadavku. Výsledek vrací controlleru. Zajišťuje komunikaci mezi jednotlivými použitými závislostmi. Uchovává závislosti
- **repository:** zajišťuje komunikaci s databází prostřednictvím adaptéru a poskytuje metody specifické pro danou doménu. Provádí překlad mezi doménovými typy a typy, které reflektují strukturu databáze. Zajišťuje kontrolu a validaci dat. Obstarává konzistentnost dat v databázi.

3.3 Frontend

Tento balíček vytváří grafické uživatelské rozhraní nad HTTP API. Slouží k prezentaci dat z API a k jejich snadnější manipulaci.

3.4 Initializer

Initializer Zajišťuje iniciální nastavení databází po startu systému. Kontejner si udržuje informace o požadovaném stavu databází. Při startu vykoná migrace PostgreSQL databáze, které nebyly aplikovány. Pro elastic search vytvoří všechny dosud neexistující indexy. Projekt je strukturován do monorepozitáře, který obsahuje několik balíčků (packages), do kterých je funkcionalita rozdělena. Kořenový balíček monorepozitáře obsahuje společnou konfiguraci nástrojů a společné balíčky.

4 CI/CD

4.1 Verzování

Pro verzování projektu jsem požil GIT a jako vzdálené úložiště GitHub. Nástroj GIT je velmi populární a v současnosti má velmi málo použitelných alternativ.

Pro vhodné použití s CI/CD pipeline, o kterém budu psát v další kapitole textu, a z důvodu lepší přehlednosti současného vývoje více funkcionalit, jsem

zavedl také konvence pro pojmenování commitů a větví. Názvy mají pevnou strukturu a přidávají entitám verzovacího systému přesnější implicitní význam. Této struktury bude v budoucnu možné využít i pro automatizaci sémantického verzování.

Níže budu popisovat jednotlivé konvence.

4.1.1 Větve

Větev obsahuje větší množství ucelených změn, které mají společný účel. Větev může být používána dlouhou dobu. Může sloužit například pro vývoj celého konkrétního modulu, pro dokumentaci, úpravy infrastruktury, nebo refactoring balíčku.

Zvláštní roli mají větve dev a main, které jsou chráněny před přímým přidáváním commitů. Změny do nich lze přidávat jen pomocí pull-requestů. Větev dev slouží ke kompletaci změn z ostatních větví. Změny, které jsou přidány do větve main jsou určeny k vydání. Vydání nové verze se provede automaticky po oštítkování commitu.

Každá větev je pojmenována ve formátu $< typ >< nazev >$, kde $< nazev >$ popisuje společný účel změn ve větvi (např. server-authentication). $< typ >$ přibližuje, jaký technický charakter změny mají (správa infrastruktury, dokumentace, testování...)

Povolené typy větví jsou následující:

- **chore:** Slouží pro údržbu projektu, závislostí, infrastruktury. Nepřidává další funkcionalitu, ani neopravuje chyby v kódu
- **feature:** Přidává skupinu funkcionalit do projektu (čtečka Bible, modlitební skupiny, ...)
- **release:** Stará se o poslední změny před vydáním nové verze. Typicky se zde provádí přidání change logu a zvýšení čísla verze.
- **docs:** Provádí se zde přidání a údržba přiložené dokumentace projektu.

4.1.2 Commity

Jednotlivé commity obsahují menší ucelené změny. Jeden commit by neměl obsahovat více změn, které spolu přímo nesouvisí (tématicky, nebo umístěním). Toto se mi nepodařilo v začátcích vývoje dodržet ve všech případech. Některé commity kromě přidání funkcionalit obsahují také refactoring, nebo opravu jiné části balíčku, která s primárními změnami přímo nesouvisí.

Pojmenování commitu je vždy ve formátu $< typ >:< popis >$, kde $< popis >$ stručně vysvětluje hlavní provedené změny v rámci daného commitu. Dále zpráva ke commitu obsahuje i podrobnější popis provedených změn.

Typy commitů jsou určeny pro budoucí implementaci automatického sémantického verzování. Usnadňují také orientaci ve změnách a upřesňují technický význam commitů. Povolené typy jsou tyto:

- **feature:** přidává novou funkcionalitu
- **fix:** opravuje chybu v kódu
- **test:** přidává automatizované testy pro již existující kód
- **docs:** upravuje dokumentaci
- **build:** změny v nastavení projektu, infrastruktuře a závislostch

4.2 CI/CD pipeline

GitHub jsem zvolil táké pro jeho rozšířenost a řadu možností, které poskytuje. mezi ně patří především služba GitHub Actions, kterou jsem použil pro vytvoření CI/CD pipeline.

GitHub Actions je flexibilní nástroj, který umožňuje spouštět akce, jako reakci na události v repozitáři. Tuto možnost používám pro spouštění akcí při přidání nových změn do vzdáleného repozitáře, kde se provedou automatizované testy a kontrola kvality kódu.

Pro commit označený štítkem se správným názvem se spustí akce, která vytvoří image jednotlivých kontejnerů projektu a přidá je do kontejnerového repozitáře.

5 Použité technologie

5.1 TypeScript

TypesCript jako hlavní jazyk, ve kterém jsem projekt psal, jsem zvolil pro jeho univerzalitu. Kompiluje se do javascriptu, který je vhodný pro použití na backendu i frontendu aplikace. Toto jsem na začátku vývoje považoval za velkou výhodu. Typescript také na rozdíl od JavaScriptu poskytuje statické typování a s ním i typovou kontrolu, což v mnoha případech značně zjednodušuje vývoj, poskytuje vhodnější automatické doplňování kódu a odhaluje řadu chyb, které by v kódu jinak vznikaly. Na mnoha místech však také kód značně znepřehledňuje a plní zbytečným přetypováním, které nemá vliv na výsledný běh kódu, vzhledem ke skutečnosti, že se vše kompiluje do javascriptu, který statické typování neimplementje. Proto je také všechny vstupy, které se do aplikace přijmou za běhu (data z databáze, HTTP požadavky, vstupy uživatele, ...), validovat znovu při běhu (k tomu jsem použil knihovnu Zod.)

5.2 Express

5.3 Vue.js

5.4 Elastic search

5.5 Docker

5.6 Github actions